

Báo Cáo Bài Tập Lập Trình - Cờ Caro AI

Môn: Cơ sở Trí Tuệ Nhân Tạo

Họ tên: Khổng Quang Huy

MSSV: 24022355

1. Mô tả bài toán cờ Caro

Trò chơi cờ Caro là một trò chơi đối kháng hai người chơi trên một bàn cờ dạng lưới vuông. Một người chơi sử dụng ký hiệu “X”, người kia sử dụng ký hiệu “O”. Mục tiêu của trò chơi là cố gắng tạo thành một đường thẳng liên tục (ngang, dọc, hoặc chéo) gồm một số lượng quân cờ nhất định để giành chiến thắng. Trong dự án này, bài toán được giới hạn với bàn cờ kích thước 9x9 và điều kiện thắng là 4 quân liên tiếp (theo tiêu chuẩn yêu cầu).

2. Luật chơi và điều kiện thắng

- Trò chơi diễn ra theo lượt. Người chơi (ký hiệu “X”) luôn được đi trước. Máy AI (ký hiệu “O”) đi sau.
- Mỗi lượt, người chơi chọn một ô trống trên bàn cờ để đánh dấu ký hiệu của mình.
- Điều kiện thắng: Trò chơi kết thúc khi một người tạo được một chuỗi 4 quân cờ của mình liên tiếp theo chiều ngang, chiều dọc, hoặc đường chéo.
- Không xét luật chặn hai đầu.
- Hòa: Nếu tất cả các ô trên bàn cờ đều đã được đánh dấu mà không ai tạo được chuỗi 4 quân, trò chơi kết thúc với kết quả hòa.

3. Cách biểu diễn trạng thái bàn cờ

Trạng thái bàn cờ được biểu diễn dưới dạng một mảng 2 chiều (ma trận/Grid) kích thước “BOARD_SIZE x BOARD_SIZE” (cụ thể là 9x9).

- Mỗi phần tử trong ma trận nhận một trong 3 giá trị hằng số: “EMPTY” (ô trống), “PLAYER_X” (ký hiệu X), hoặc “PLAYER_O” (ký hiệu O).
- Cấu trúc dữ liệu “Board” được thiết kế theo hướng đối tượng (OOP) để dễ dàng sao chép (copy) cấu hình bàn cờ nhằm phục vụ cho việc giả lập nước đi trong thuật toán tìm kiếm của AI.

4. Cách sinh nước đi hợp lệ

Để tối ưu hóa quá trình duyệt cây trò chơi của AI và tránh việc duyệt dư thừa trên những vùng không gian bàn cờ trống rỗng, thuật toán sinh nước đi hợp lệ (candidate generation) được áp dụng:

- Thuật toán chỉ xét các ô trống nằm trong vùng lân cận (với bán kính “CANDIDATE_RADIUS = 1”) xung quanh các ô đã có quân cờ.
- Điều này có nghĩa là AI sẽ chỉ thử nghiệm những nước đi nằm cạnh các quân cờ đang tồn tại, giúp giảm thiểu tối đa "Hệ số phân nhánh" (Branching factor) của cây trò chơi, tiết kiệm tài nguyên mà vẫn không làm lỡ các nước đi phòng thủ hoặc tấn công chiến thuật.

5. Cách kiểm tra trạng thái kết thúc

Hàm kiểm tra trò chơi kết thúc hoạt động tối ưu bằng cách chỉ kiểm tra xung quanh vị trí của nước đi cuối cùng vừa được đánh.

- Từ vị trí ô này, thuật toán sẽ đếm số lượng quân cờ giống nhau liên tiếp trải dài theo 4 hướng:
 1. Chiều ngang (Trái - Phải)
 2. Chiều dọc (Lên - Xuống)
 3. Đường chéo chính (Tây Bắc - Đông Nam)
 4. Đường chéo phụ (Đông Bắc - Tây Nam)
- Nếu ở bất kỳ hướng nào tổng số quân liên tiếp cộng dồn lại đạt “WIN_LENGTH” (bằng 4), người vừa đánh sẽ được ghi nhận là chiến thắng. Trò chơi cũng kiểm tra xem bảng cờ đã kín ô trống hay chưa để đưa ra kết quả hòa.

6. Thuật toán Minimax đã cài đặt

Thuật toán Minimax được cài đặt thông qua phương pháp đệ quy sâu.

- Tại mỗi node, nếu đạt đến “MAX_DEPTH” (độ sâu tối đa giới hạn) hoặc trò chơi kết thúc, hàm trả về giá trị đánh giá (heuristic score) của bàn cờ đó.
- Lượt của AI (Max Player): Thuật toán mô phỏng việc AI sẽ duyệt qua tất cả các nước đi hợp lệ, đệ quy gọi Minimax cho lượt của đối thủ, và chọn ra nước đi mang lại điểm số cao nhất cho AI.
- Lượt của người chơi (Min Player): Thuật toán mô phỏng việc người chơi (đối thủ của AI) sẽ luôn có những lựa chọn tối ưu, chọn ra nước đi kéo điểm số đánh giá xuống mức thấp nhất, gây bất lợi nhất cho AI.

7. Thuật toán Alpha-Beta đã cài đặt

Thuật toán Alpha-Beta Pruning (Cắt tỉa Alpha-Beta) được cài đặt như một bản tối ưu hóa mạnh mẽ cho Minimax nhằm giảm bớt các nhánh duyệt dư thừa.

- Hàm truyền thêm hai tham số: “alpha” (giá trị điểm tốt nhất hiện tại mà nhánh Max có thể đảm bảo) và “beta” (giá trị tồi nhất hiện tại mà nhánh Min có thể gây ra).
- Trong lượt của AI (Max), “alpha” liên tục được cập nhật. Nếu giá trị điểm tính được lớn hơn hoặc bằng “beta”, nhánh đệ quy hiện tại lập tức dừng (break - cắt tỉa nhánh), vì đối thủ sẽ không bao giờ cho phép AI đi đến nhánh này.
- Tương tự trong lượt Min, “beta” được cập nhật, và xảy ra cắt nhánh nếu điểm số nhỏ hơn hoặc bằng “alpha”.

8. Hàm đánh giá trạng thái

Hàm đánh giá là cốt lõi để AI đánh giá thế trận có lợi hay không khi chưa phân định kết quả thắng thua.

- Thuật toán quét và ghi nhận các chuỗi quân cờ dọc theo các hướng ngang, dọc, chéo.
- Dựa trên độ dài chuỗi và độ mở ở hai đầu, điểm số được gán (Ví dụ: 4 quân liên tiếp có điểm cực lớn; 3 quân liên tiếp ở hai đầu được cộng hàng nghìn điểm để phòng thủ/tấn công; 3 quân liên tiếp chặn một đầu hoặc 2 quân ở hai đầu được điểm thấp hơn).
- Điểm tổng của bàn cờ là hiệu số của “Tổng điểm thế cờ của AI” trừ đi “Tổng điểm thế cờ của Người chơi”. Điểm dương nghĩa là AI đang chiếm ưu thế.

9. Thiết kế các trạng thái thử nghiệm

Để so sánh khoa học hiệu năng giữa thuật toán Minimax và Alpha-Beta, tập hợp các "Trạng thái Benchmark" (BENCHMARK_STATES) được thiết lập với 5 trạng thái cụ thể:

- Opening (Trạng thái đầu ván): Bàn cờ gần như trống, chỉ có 1 quân cờ. Đánh giá khả năng xử lý khi hệ số phân nhánh lớn nhất ở giai đoạn đầu game.
- Midgame (Trạng thái giữa ván): Thế trận bắt đầu hình thành với một vài quân cờ của cả hai bên. Đánh giá khả năng phát triển thế cờ cơ bản.
- AI Can Win (AI có thể thắng ngay): AI (quân O) đã có sẵn 3 quân liên tiếp. Đánh giá khả năng AI nhận diện và chớp thời cơ để giành chiến thắng ngay lập tức.

- Must Block (Bắt buộc phải chặn): Người chơi (quân X) đã có 3 quân liên tiếp đang đe dọa. Đánh giá khả năng AI phát hiện nguy hiểm và ưu tiên đánh nước phòng thủ để chặn đối phương.
- Mutual Attack (Hai bên cùng tấn công): Thế trận giằng co phức tạp, cả hai bên đều đang có những chuỗi quân đe dọa lẫn nhau. Đánh giá mức độ tìm kiếm sâu và khả năng đưa ra quyết định tối ưu giữa việc tấn công hay phòng thủ.

10. Bảng kết quả thực nghiệm

PS C:\Users\KHONG QUANG HUY\24022355_KhongQuangHuy_CaroAI> & "c:/Users/KHONG QUANG HUY/24022355_KhongQuangHuy_CaroAI/.venv/Scripts/python.exe" "c:/Users/KHONG QUANG HUY/24022355_KhongQuangHuy_CaroAI/src/benchmark/runner.py"

BẮT ĐẦU CHẠY BENCHMARK SO SÁNH MINIMAX VÀ ALPHA-BETA

Kết quả Thực nghiệm							
Trạng thái	Depth	Thuật toán	Nước đi	Điểm	Số trạng thái xét	Thời gian (ms)	Nhận xét
Opening	1	Minimax	(5, 5)	0.0	9	0.60	Giảm 0.0% (Khớp)
	1	Alpha-Beta	(5, 5)	0.0	9	0.48	
Opening	2	Minimax	(5, 5)	-110.00000000000001	97	4.96	Giảm 66.0% (Khớp)
	2	Alpha-Beta	(5, 5)	-110.00000000000001	33	1.47	
Opening	3	Minimax	(5, 5)	-10.000000000000014	1297	69.61	Giảm 72.6% (Khớp)
	3	Alpha-Beta	(5, 5)	-10.000000000000014	356	20.68	
Midgame	1	Minimax	(6, 5)	9979.0	17	1.18	Giảm 0.0% (Khớp)
	1	Alpha-Beta	(6, 5)	9979.0	17	1.33	
Midgame	2	Minimax	(5, 5)	-811.0	307	20.68	Giảm 73.9% (Khớp)
	2	Alpha-Beta	(5, 5)	-811.0	80	5.29	
Midgame	3	Minimax	(6, 5)	10000000	6173	372.33	Giảm 88.2% (Khớp)
	3	Alpha-Beta	(6, 5)	10000000	729	42.60	
AI Can Win	1	Minimax	(7, 4)	10000000	13	0.58	Giảm 0.0% (Khớp)
	1	Alpha-Beta	(7, 4)	10000000	13	0.60	
AI Can Win	2	Minimax	(7, 4)	10000001	161	8.42	Giảm 85.7% (Khớp)
	2	Alpha-Beta	(7, 4)	10000001	23	1.28	
AI Can Win	3	Minimax	(7, 4)	10000002	2687	136.31	Giảm 93.0% (Khớp)
	3	Alpha-Beta	(7, 4)	10000002	187	10.35	
Must Block	1	Minimax	(2, 3)	-1100.0	17	0.86	Giảm 0.0% (Khớp)
	1	Alpha-Beta	(2, 3)	-1100.0	17	1.00	
Must Block	2	Minimax	(4, 4)	-10000000	305	15.68	Giảm 35.4% (Khớp)
	2	Alpha-Beta	(4, 4)	-10000000	197	9.47	

Problems	Output	Debug Console	Terminal	Ports			
Opening		3	Minimax	(5, 5)	-10.00000000000014	1297	69.61
		3	Alpha-Beta	(5, 5)	-10.00000000000014	356	20.68
Midgame		1	Minimax	(6, 5)	9979.0	17	1.18
		1	Alpha-Beta	(6, 5)	9979.0	17	1.33
Midgame		2	Minimax	(5, 5)	-811.0	307	20.68
		2	Alpha-Beta	(5, 5)	-811.0	80	5.29
Midgame		3	Minimax	(6, 5)	1000000	6173	372.33
		3	Alpha-Beta	(6, 5)	1000000	729	42.60
AI Can Win		1	Minimax	(7, 4)	1000000	13	0.58
		1	Alpha-Beta	(7, 4)	1000000	13	0.60
AI Can Win		2	Minimax	(7, 4)	1000001	161	8.42
		2	Alpha-Beta	(7, 4)	1000001	23	1.28
AI Can Win		3	Minimax	(7, 4)	1000002	2687	136.31
		3	Alpha-Beta	(7, 4)	1000002	187	10.35
Must Block		1	Minimax	(2, 3)	-1100.0	17	0.86
		1	Alpha-Beta	(2, 3)	-1100.0	17	1.00
Must Block		2	Minimax	(4, 4)	-1000000	305	15.68
		2	Alpha-Beta	(4, 4)	-1000000	197	9.47
Must Block		3	Minimax	(4, 4)	-1000001	5406	308.88
		3	Alpha-Beta	(4, 4)	-1000001	2332	127.36
Mutual Attack		1	Minimax	(5, 4)	10299.0	21	1.22
		1	Alpha-Beta	(5, 4)	10299.0	21	1.23
Mutual Attack		2	Minimax	(2, 3)	1088.0	461	29.37
		2	Alpha-Beta	(2, 3)	1088.0	282	16.80
Mutual Attack		3	Minimax	(5, 4)	1000000	10960	677.02
		3	Alpha-Beta	(5, 4)	1000000	900	53.85

PS C:\Users\KHONG QUANG HUY>24022355_KhongQuangHuy_CarpATS

Trong quá trình thực nghiệm và hiển thị bảng kết quả so sánh Minimax và Alpha-Beta, cột "Điểm" thể hiện giá trị đánh giá (Heuristic value) hoặc kết quả thực tế của nhánh cây trò chơi mà AI đã duyệt được. Điểm càng dương thì AI (Player O) càng chiếm lợi thế, điểm càng âm thì Người chơi (Player X) càng chiếm lợi thế.

Dưới đây là tổng hợp toàn bộ các trường hợp và cơ chế tính điểm trong hệ thống :

1. Trạng Thái Cờ Đã Kết Thúc

- AI Chắc Chắn Thắng: “Điểm = 10.000.000 + depth”

- Hệ thống thưởng số điểm cực lớn (10 triệu) vượt xa mọi đánh giá thế cờ bình thường để đảm bảo AI sẽ chọn nước cờ mang lại chiến thắng.
- Ý nghĩa của việc cộng thêm “depth”: Giúp AI "ưu tiên thắng nhanh". Tìm thấy đường thắng ở độ sâu còn lại lớn (tức tốn ít nước đi hơn) sẽ mang lại điểm cao hơn so với rề rà tốn nhiều nước mới thắng. Nhờ vậy AI không vờn người chơi mà sẽ chốt hạ luôn.

- Người Chơi Chắc Chắn Thắng (AI Thua): “Điểm = -10.000.000 - depth”

- Điểm số bị phạt cực nặng (âm 10 triệu).
 - Ý nghĩa của việc trừ đi `depth`: Giúp AI "trì hoãn thất bại". Nếu việc thua là không thể tránh khỏi, AI sẽ chọn nhánh đi kéo dài thời gian sống sót lâu nhất có thể (độ sâu "depth" tiến về nhỏ nhất thì điểm sẽ đỡ bị âm sâu hơn).
- Trạng Thái Hòa: "Điểm = 0"
- Bàn cờ đã đầy nhưng không ai có 4 quân liên tiếp.

2. Trạng Thái Cờ Chưa Kết Thúc

- Khi AI đã duyệt đến hết giới hạn số bước đi cho phép mà ván cờ vẫn chưa ngã ngũ, nó sẽ dùng hàm đánh giá tĩnh để phỏng đoán độ lợi thế của bàn cờ.
- AI sẽ trích xuất tất cả các đường thẳng (ngang, dọc, 2 đường chéo) và chấm điểm theo như cách tính trong file "src/ai/evaluate.py".

Giải thích việc nhân 1.1 cho Người Chơi :

- Việc này đóng vai trò giúp AI thiên về phòng thủ. Khi điểm của Người Chơi bị khuếch đại nhẹ (nhân 1.1), AI sẽ đánh giá mỗi đe dọa từ thế tấn công của người chơi nguy hiểm hơn bình thường, từ đó chủ động ưu tiên đi nước chặn hơn là cố gắng tấn công một cách mù quáng.

Tóm Lại :

- Nếu cột điểm hiển thị là một số lẻ xung quanh mốc $\sim 10.000.000$ hoặc $\sim -10.000.000$ (vd: 10000002 hay -10000001), điều đó có nghĩa là ở trạng thái board game đó, với mức "depth" tương ứng, AI đã tính toán thành công đến kết quả cuối cùng (thắng/thua).
- Nếu cột điểm hiển thị ở các giá trị nhỏ hơn (vài trăm ngàn, vài chục ngàn, vd: 10210 hoặc -5400), thì ván cờ vẫn ở giai đoạn giằng co và đây chỉ là điểm số phỏng đoán từ hàm evaluate().

11. Nhận xét về số trạng thái đã xét và thời gian chạy

- Số lượng node đã duyệt: Thuật toán Alpha-Beta cho thấy sự vượt trội tuyệt đối về khả năng cắt tỉa nhánh cây dư thừa. Số lượng trạng thái xét của Alpha-Beta luôn chỉ bằng một phần nhỏ (thường giảm từ 50% đến hơn 80%) so với Minimax.
- Thời gian chạy: Nhờ lượng trạng thái cần duyệt giảm mạnh, thời gian tính toán của Alpha-Beta diễn ra nhanh hơn rất nhiều, đáp ứng tốt cho giao diện thời gian

thực. Tốc độ này đặc biệt rõ ràng trên các bàn cờ có khoảng trống rộng và khi tăng độ sâu.

- Trong mọi bài test, điểm số và nước đi cuối cùng được chọn bởi Alpha-Beta và Minimax luôn khớp nhau, minh chứng rằng phương pháp cắt tỉa an toàn tuyệt đối và không làm suy giảm chất lượng nước cờ.

12. Nhận xét về ảnh hưởng của độ sâu tìm kiếm

- Độ sâu (Depth) 1 hoặc 2: AI phản ứng ngay lập tức nhưng có tầm nhìn khá nông, thường chỉ ưu tiên chặn đứng ngay các nguy hiểm trực diện mà chưa thể giăng bẫy đối phương.

- Độ sâu từ 3 trở lên: AI thông minh hơn, có thể dự đoán chuỗi hành động và tự xây dựng các tổ hợp bẫy (như đánh nước đôi). Tuy nhiên, độ sâu lớn sẽ đẩy số lượng node của thuật toán Minimax tăng theo cấp số nhân, làm game lag; trong khi Alpha-Beta vẫn duy trì được thời gian chờ hợp lý.

13. Ưu điểm và hạn chế của chương trình

Ưu điểm:

- Cấu trúc source code rõ ràng, phân tầng khéo léo (MVC/Module hóa), tích hợp benchmark công cụ.
- Trải nghiệm giao diện (GUI) thân thiện qua “pygame”, hỗ trợ đối trực tiếp thuật toán thi đấu khi đang ở Menu màn hình chờ.
- Tích hợp thành công thuật toán Cắt tỉa Alpha-Beta và giới hạn không gian duyệt nước đi khả dĩ (“CANDIDATE_RADIUS”), giúp AI Caro có thể phản xạ nhanh chóng.

Hạn chế:

- Hàm đánh giá (Heuristic Evaluation Function) còn ở mức cơ bản, nếu gặp đối thủ thực sự giỏi với các thế bài gomoku nâng cao, AI có thể bị lừa. Hàm đánh giá chưa xét tổ hợp mối đe dọa (ví dụ: AI có hai chuỗi 3 mở cùng lúc — thực tế gần như thắng chắc, nhưng hàm hiện tại chỉ cộng điểm đơn lẻ từng chuỗi).
- “CANDIDATE_RADIUS” cố định = 1 có thể bỏ sót nước đi tốt ở xa trong giai đoạn đầu ván.
- Để đánh ở độ sâu cao hơn, thuật toán cơ bản vẫn sẽ gặp giới hạn thời gian thực tế, sẽ càng tốn nhiều thời gian ở các độ sâu cao hơn.

14. Tài liệu tham khảo và phần đã tham khảo từ các repo mẫu

- Tài liệu tham khảo:

- 1 số video về cách hoạt động của thuật toán Minimax và Alpha-Beta Pruning trong trò chơi caro.
- Các video về việc hướng dẫn sử dụng thư viện pygame

- Repo mẫu được cung cấp:

- gomokuAI-py: Tham khảo cách tổ chức project Python, cách biểu diễn bàn cờ bằng mảng 2D, cách tách module game/AI/UI và cách viết script đo thời gian theo độ sâu.
- Caro_AI: Tham khảo ý tưởng tổng quát về candidate move generation, cách ghi log kết quả benchmark và cấu trúc module config.